

智能学习助手与知识管理多智能体系统 —— 软件体系结构设计文档

目录

- [1. 用户需求 \(User Requirements\)](#)
- [2. 功能分解 \(Functional Decomposition\)](#)
- [3. 软件体系风格设计 \(Software Architecture Style\)](#)
- [4. 智能体设计模式分析 \(Agent Design Patterns\)](#)
- [5. 4+1 视图分析 \(4+1 View Model\)](#)
- [6. Demo 系统实现说明](#)
- [7. 个人分工说明](#)

一、用户需求

1.1 应用场景与目标用户

应用场景：面向高校学生的智能学习助手系统。学生在日常学习中积累了大量课件（PDF、笔记、Markdown等格式），但面临以下痛点：

- 课件分散、信息检索困难，考试前不知道从何复习
- 缺乏个性化的学习规划和进度跟踪
- 无法快速检验自己对知识点的掌握程度
- 手工整理笔记耗时费力

本系统通过**多智能体协作**架构，将这些学习资源整合为知识库，提供智能问答、学习规划、知识总结、自测练习等一站式学习辅助服务。

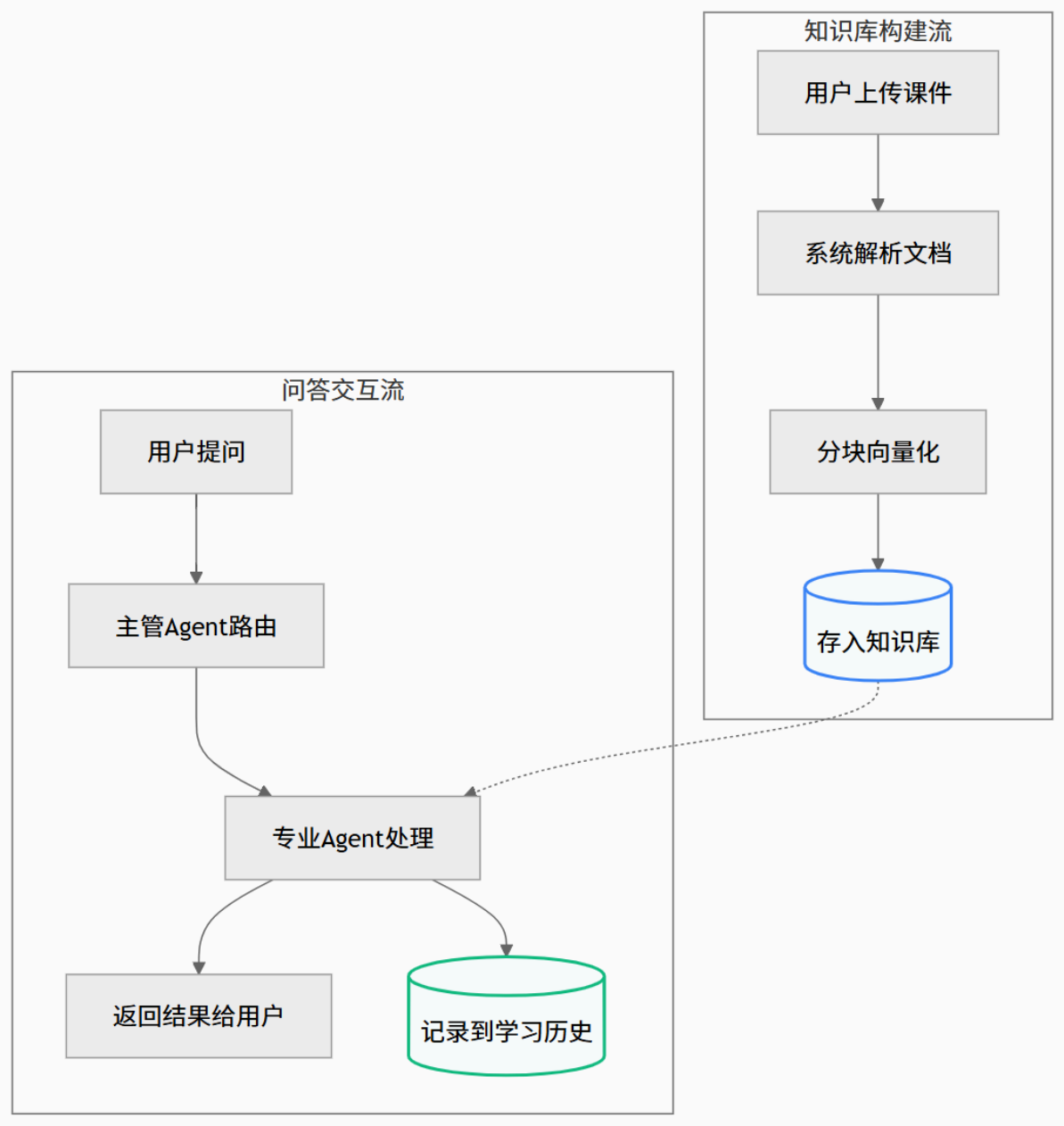
目标用户：高校学生（本科生/研究生），尤其是面临期中、期末考试复习压力的学生群体。

1.2 功能性需求

编号	功能模块	功能描述	优先级
FR-01	文档管理	支持上传 PDF、Markdown、TXT 格式课件；支持查看、删除已上传文档	高
FR-02	文档解析与向量化	自动解析上传文档内容，进行文本分块(Chunking)和向量嵌入(Embedding)，存入知识库	高
FR-03	智能问答	基于已上传课件内容，通过RAG回答学生的学业问题，给出引用来源	高
FR-04	学习规划	接收考试科目与时间，自动生成分阶段的复习计划，包含每日学习任务	中

编号	功能模块	功能描述	优先级
FR-05	知识总结	对选定的课件或知识点进行结构化总结，生成层次化笔记大纲	中
FR-06	自测练习	根据课件知识点自动生成选择题或简答题，支持答题与即时反馈	中
FR-07	学习记录	记录问答历史、学习进度、测验成绩，提供学习报告	低

核心交互流程：



1.3 非功能性需求

编号	类别	需求描述	指标
NFR-01	可扩展性	新类型的专业Agent应能方便地注册到系统中，无需修改核心框架	新增Agent ≤ 50行代码改动

编号	类别	需求描述	指标
NFR-02	响应性	普通问答应在可接受时间内返回结果	问答响应 ≤ 10秒
NFR-03	易用性	用户界面直观简洁，非技术人员可快速上手	首次使用 ≤ 3步即可开始
NFR-04	模块化	各模块职责单一、接口清晰，支持独立测试和替换	模块间仅通过定义好的接口通信
NFR-05	可靠性	系统应优雅处理LLM调用失败、文档解析错误等异常	关键操作有重试和降级机制
NFR-06	安全护栏	系统应拒绝对考试作弊、侵权等不当请求的响应	内置安全检测机制

二、功能分解

2.1 顶层功能模块划分

系统划分为**六大功能模块**，通过清晰的接口协作：



2.2 子模块职责与接口

2.2.1 文档管理模块

属性	说明
职责	负责课件的上传、存储、解析与向量化

属性	说明
输入接口	<code>upload_document(file: bytes, filename: str) → Document</code>
输出接口	<code>list_documents() → List[Document]</code> , <code>delete_document(doc_id: str) → bool</code>
依赖	知识库模块（用于存储解析后的文档块和向量）

内部子流程（管道-过滤器模式）：



2.2.2 智能问答模块

属性	说明
职责	基于知识库中课件内容，通过RAG回答学生问题
输入接口	<code>ask_question(query: str, history: List[Message]) → Answer</code>
输出接口	结构化回答（含答案文本 + 引用来源 + 置信度）
依赖	知识库模块（检索相关文档块）、LLM服务

2.2.3 学习规划模块

属性	说明
职责	根据考试日期和科目，生成个性化分阶段复习计划
输入接口	<code>generate_plan(subject: str, exam_date: date, knowledge_points: List[str]) → StudyPlan</code>
输出接口	结构化学习计划（含每日任务、里程碑、预计时长）
依赖	LLM服务

2.2.4 知识总结模块

属性	说明
职责	提取课件关键知识点，生成结构化笔记/思维导图大纲
输入接口	<code>summarize(doc_ids: List[str], style: str) → Summary</code>
输出接口	层次化大纲（Markdown格式，含标题层级、关键概念、公式）
依赖	知识库模块（获取文档内容）、LLM服务

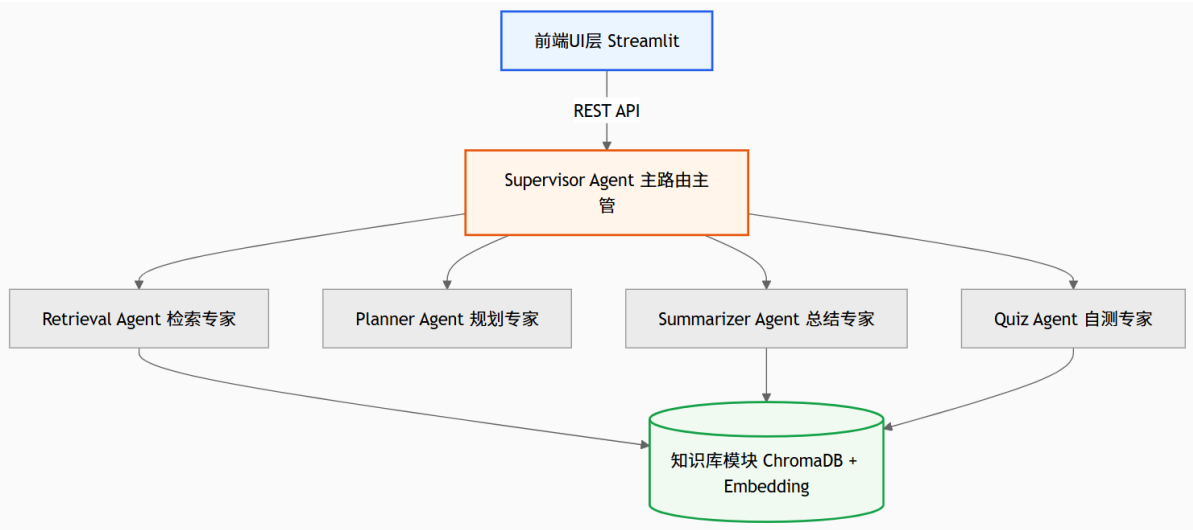
2.2.5 自测练习模块

属性	说明
职责	根据知识点自动生成练习题，评估学生掌握程度
输入接口	<code>generate_quiz(doc_ids: List[str], question_count: int, question_type: str) → Quiz</code>
输出接口	题目列表（含题干、选项、正确答案、解析）
依赖	知识库模块（获取知识点内容）、LLM服务

2.2.6 知识库模块

属性	说明
职责	存储文档块和向量，提供语义检索能力
输入接口	<code>add_documents(chunks: List[DocumentChunk], embeddings: List[Vector]) → bool</code>
输出接口	<code>semantic_search(query: str, top_k: int) → List[RelevantChunk]</code>
技术栈	ChromaDB（向量存储）+ Sentence Transformers（嵌入模型）

2.3 模块间依赖关系



依赖方向：单向依赖，上层依赖下层。前端UI → Supervisor Agent → 各专业Agent → 知识库。不存在循环依赖。

三、软件体系风格设计

3.1 风格选择概述

本系统采用三种经典软件体系结构风格的组合：

- 1. 分层风格 (Layered Style) — 作为系统的主要组织风格
- 2. 数据为中心风格 (Data-Centered Style) — 知识库作为核心共享数据源
- 3. 管道-过滤器风格 (Pipe-Filter Style) — 文档处理流水线

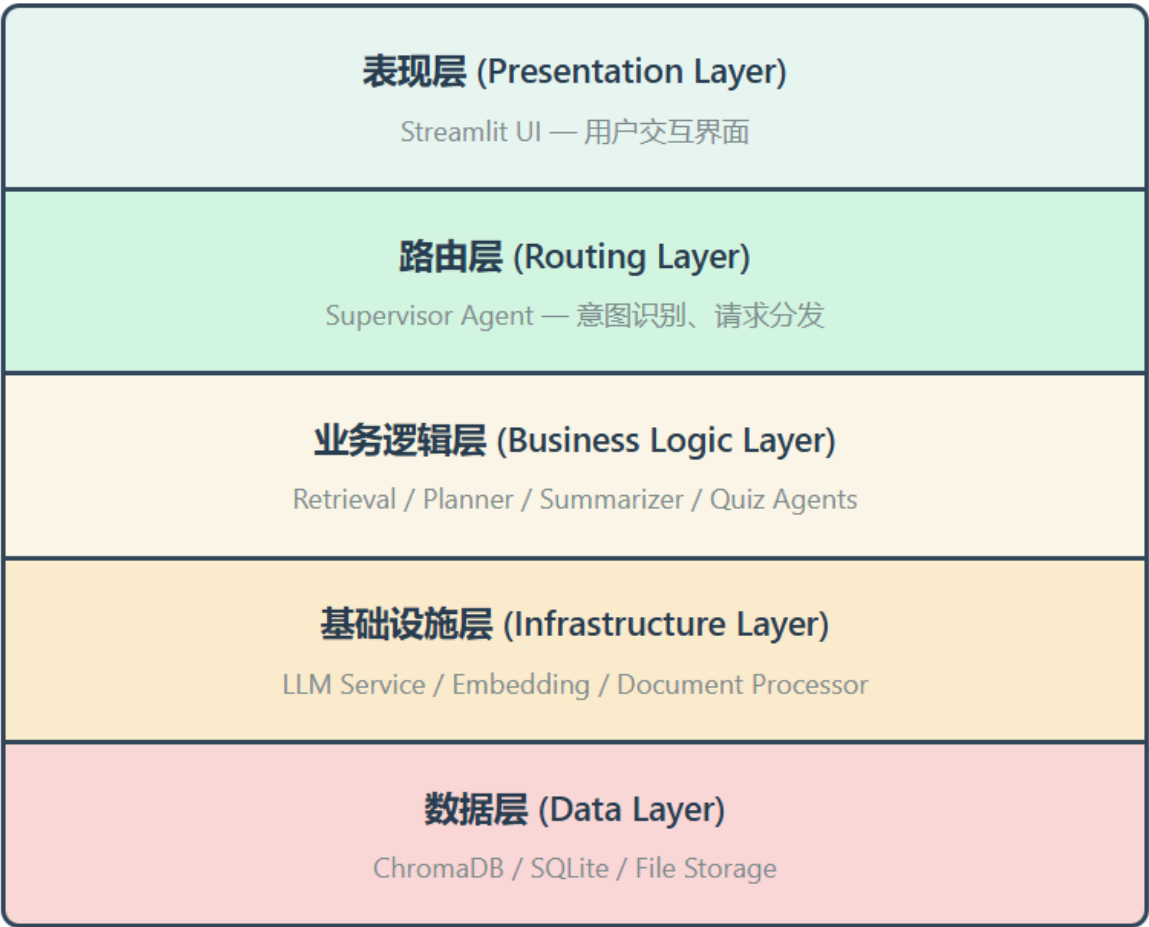
3.2 分层风格（主风格）

选择理由：

分层风格将系统按抽象层次垂直划分为若干层，每层只依赖其直接下层。这天然适合本系统的Web-Agent架构：

- 表现层负责用户交互，可独立替换（如从Streamlit换为Web App）
- 业务逻辑层（Agent层）封装智能体决策逻辑，与表现层解耦
- 数据层封装存储细节，可替换底层数据库

层次划分：



分层风格优缺点分析：

优点	缺点	本系统中的适配
每层可独立开发和测试	层间调用可能有性能开销	系统规模适中，开销可接受
支持逐层替换（如换前端框架）	严格分层可能导致"穿透调用"问题	定义清晰的层间API接口
降低系统复杂度，便于理解	不是所有功能都能干净地归属到某一层	Agent层已是良好抽象

分层约束：

- 上层只能调用相邻下层（表现层 → 路由层 → 业务逻辑层 → 基础设施层 → 数据层）
- 跨层通信通过事件机制实现（如数据层变更通知业务层）
- 每层只暴露接口，隐藏实现细节

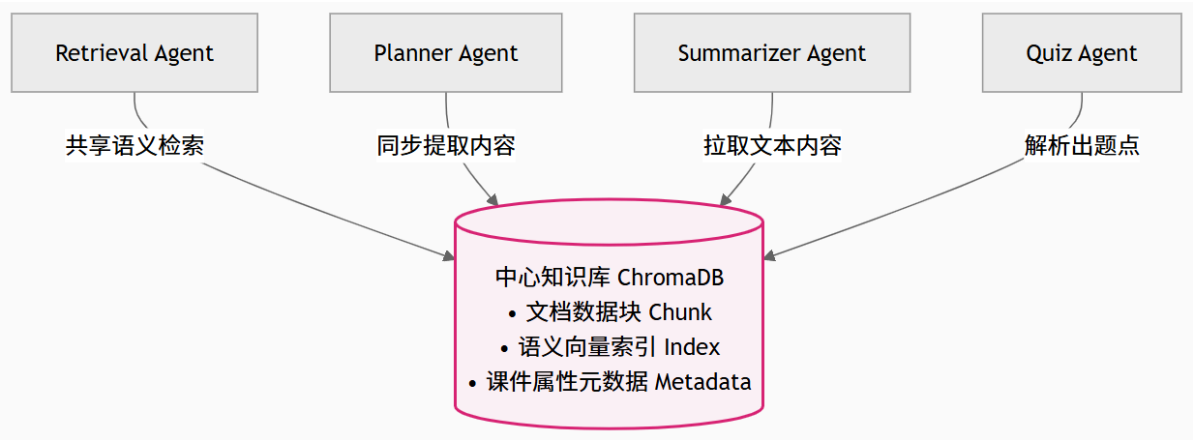
3.3 数据为中心风格

选择理由：

本系统的核心价值在于"基于课件知识的智能服务"，知识库是所有Agent的共享核心。数据为中心风格的特征——多个独立组件通过共享数据源通信——完美契合RAG架构：

- ChromaDB 向量数据库是中心数据仓库
- 各专业Agent独立运行，通过检索同一知识库获取所需上下文
- Agent之间不直接通信，而是通过知识库的状态变化间接协调
- 新增Agent只需接入知识库接口，无需修改其他Agent

在系统中的应用：



数据为中心风格优缺点分析：

优点	缺点	本系统中的适配
Agent间松耦合，易于扩展	共享数据的一致性需保证	文档写入有事务保护
数据持久化，Agent可独立重启	中心数据源是单点瓶颈	ChromaDB支持持久化和并发读
新Agent接入成本低	数据模型变更影响所有Agent	使用统一Document Schema

3.4 管道-过滤器风格

选择理由：

文档处理是一个典型的ETL（Extract-Transform-Load）流水线场景。管道-过滤器风格将处理过程分解为一系列独立的过滤器（Filter），通过管道（Pipe）传递数据。每一步只关注自己的转换逻辑，不关心上下游具体实现。

在系统中的应用——文档处理流水线：



各Filter详细职责：

过滤器	输入	输出	职责
格式检测	原始文件字节	FileType + 文件元数据	识别文件类型(PDF/MD/TXT)
文本提取	FileType + 字节	原始文本字符串	根据类型调用对应解析器
文本清洗	原始文本	清洗后文本	去除乱码、格式符号、冗余空行
智能分块	清洗文本	文本块列表	按语义边界切分，控制块大小
向量嵌入	文本块列表	文本块+向量	调用Embedding模型生成向量
存入DB	文本块+向量	成功/失败状态	写入ChromaDB并建立索引

管道-过滤器风格优缺点分析：

优点	缺点	本系统中的适配
过滤器可独立复用和测试	不适合需要交互的场景	文档处理是单向批量处理
流水线可灵活重组	批量处理有延迟	文档上传是低频操作
支持并行处理	中间数据格式需统一约定	统一使用DocumentChunk数据结构

3.5 风格组合总结

三种风格在系统中**互补协作**，各自负责不同层面的架构组织：

风格	组织维度	作用范围
分层风格	垂直层次	系统整体从前端到数据的完整调用链
数据为中心	水平共享	Agent间通过知识库间接协作
管道-过滤器	处理流水线	文档从上传到入库的转换链路

四、智能体设计模式分析

4.1 路由模式 (Routing)

模式定义

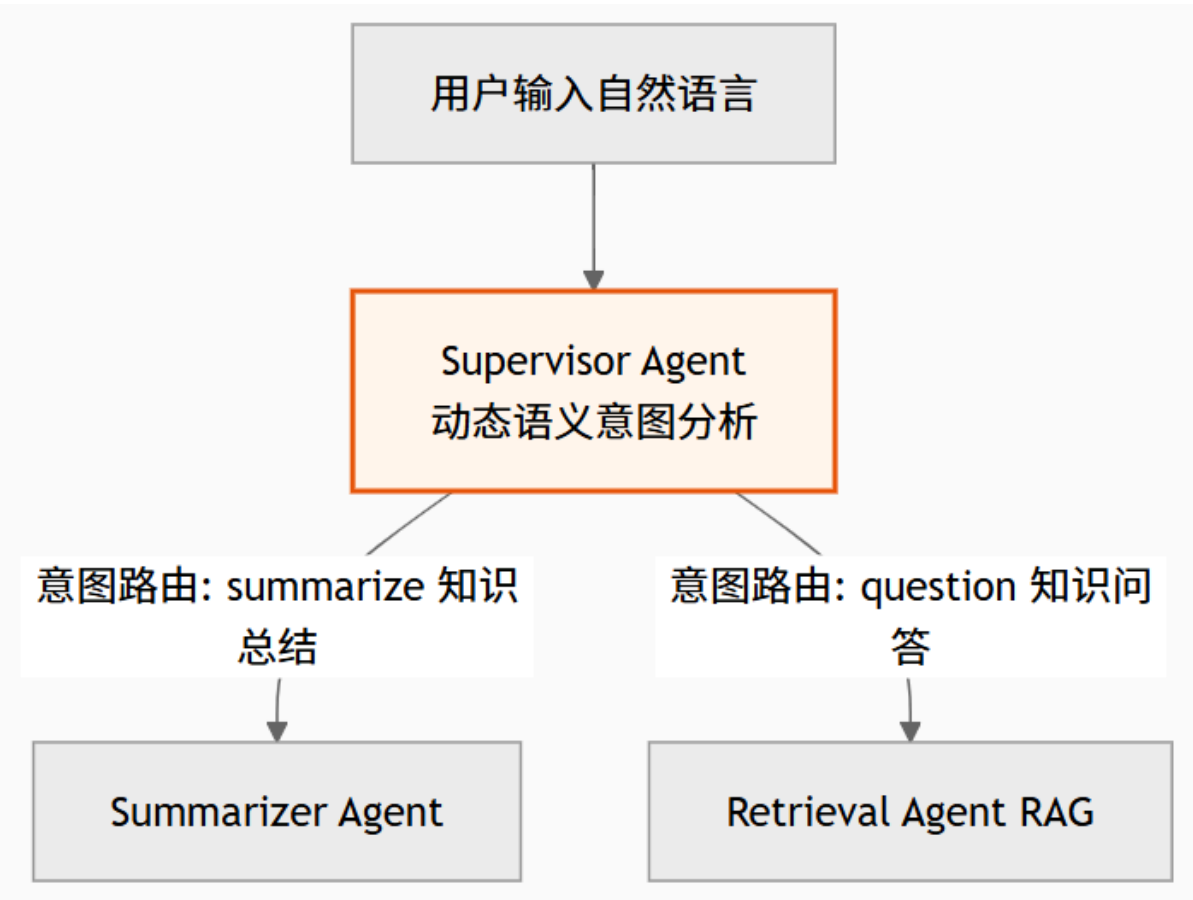
路由模式使智能体系统能够根据环境状态、用户输入或前序操作结果，在多个潜在动作之间动态决策，将控制流导向不同的专用函数、工具或子流程。该模式将系统从固定执行路径转变为动态评估的灵活系统。

在本系统中的应用

应用位置： Supervisor Agent（监督者Agent）

应用方式：

Supervisor Agent 作为系统的入口路由，接收所有用户请求，通过LLM分析用户意图，然后将请求路由到对应的专业Agent：



路由决策依据：

- 用户输入的自然语言语义分析
- 对话历史上下文（如用户刚才上传了文档，可能更倾向于提问）
- 关键词匹配与意图分类（系统预定义5种意图：question / summarize / plan / quiz / upload）

作用与价值：

1. **关注点分离：** 每个专业Agent只需关注自己的领域，无需处理意图识别逻辑
2. **可扩展性：** 新增Agent类型只需在路由表中注册新的意图映射
3. **用户体验：** 用户无需手动切换功能模式，系统自动理解意图
4. **容错能力：** 当意图不明确时，Supervisor可以反问用户确认（降级策略）

4.2 知识检索模式 (RAG - Retrieval Augmented Generation)

模式定义

RAG (检索增强生成) 让LLM能够访问外部、最新、专有的信息。该模式包含两个核心步骤：检索（从知识库中搜索与查询相关的片段）和增强（将检索到的片段加入LLM提示词中）。RAG帮助LLM克服训练数据过时的问题，减少"幻觉"，实现领域知识的精确集成。

在本系统中的应用

应用位置： Retrieval Agent (检索智能体)

应用方式：

1 查询向量化

将用户问题转为向量: [0.023, -0.451, 0.782, ...]

2 语义检索

在ChromaDB中搜索Top-K相关文档块

- Chunk#42: "软件体系结构由构件、连接件和约束组成..." (相似度: 0.92)
- Chunk#87: "软件体系结构的定义分为组成派和决策派..." (相似度: 0.87)
- Chunk#15: "软件体系结构风格是指..." (相似度: 0.81)

3 上下文增强

构建Prompt，将检索结果加入上下文：

"请基于以下参考资料回答用户问题。

参考资料1: ...(Chunk#42内容)

参考资料2: ...(Chunk#87内容)

用户问题: 软件体系结构的定义是什么？

请用中文回答，并标注引用来源。"

4 LLM生成

LLM生成带引用的答案 → 返回用户

在系统中的RAG特性：

- **来源可追溯：** 每个回答标注引用的课件来源和段落，学生可核实
- **领域精准：** 答案严格基于用户上传的课件，而非LLM的通用知识
- **抗幻觉：** 通过强制LLM基于检索内容回答，显著减少编造内容
- **动态更新：** 新增课件后，问答自动覆盖新知识

作用与价值：

1. 将通用LLM转变为特定课程的领域专家

2. 答案有据可查，符合学术诚信要求
3. 降低LLM"幻觉"对学生学习的误导风险
4. 支持私有知识库——课件不泄露给外部

4.3 提示链模式 (Prompt Chaining)

模式定义

提示链 (Prompt Chaining)，也称为流水线模式。它将单一、复杂的提示分解为一系列步骤，每一步的LLM输出作为下一步的输入。该模式解决了复杂任务中LLM容易忽略指令、丢失上下文、错误累积等问题，提升了多步骤任务处理的可靠性和可控性。

在本系统中的应用

应用位置： Summarizer Agent (知识总结智能体)

应用方式：

知识总结任务如果用一个Prompt直接要求LLM"总结所有内容"，往往会导致：

- 对长文档超出上下文窗口
- 遗漏重要细节
- 结构混乱

因此采用**三阶段提示链**：

【阶段1: 关键点提取】

"请从以下课件内容中提取15-20个关键知识点，
每个知识点一行，用编号列出。
课件内容：{chunk_texts}"

↓ 输出: 知识点列表

【阶段2: 结构化归纳】

"请将以下知识点按主题分组，为每组生成
一个标题，形成层次结构。
知识点列表：{stage1_output}"

↓ 输出: 分组大纲

【阶段3: 精细化输出】

"请将以下大纲扩展为完整的学习笔记，每个
知识点包含：定义、关键要点、示例。
大纲：{stage2_output}"

↓ 输出: 完整结构化笔记 (Markdown格式)

链的优势：

1. **降低单次LLM调用复杂度**：每阶段有明确的小目标
2. **中间结果可见可检查**：学生可查看各阶段输出，确认无误
3. **灵活配置**：每阶段可选不同模型（如阶段3用更强模型做精细化）
4. **可控质量**：可在阶段间插入人工审核

作用与价值：

1. 提升长文档总结质量和完整性
2. 中间步骤可见，消除"黑箱"感
3. 每阶段可独立调试和优化

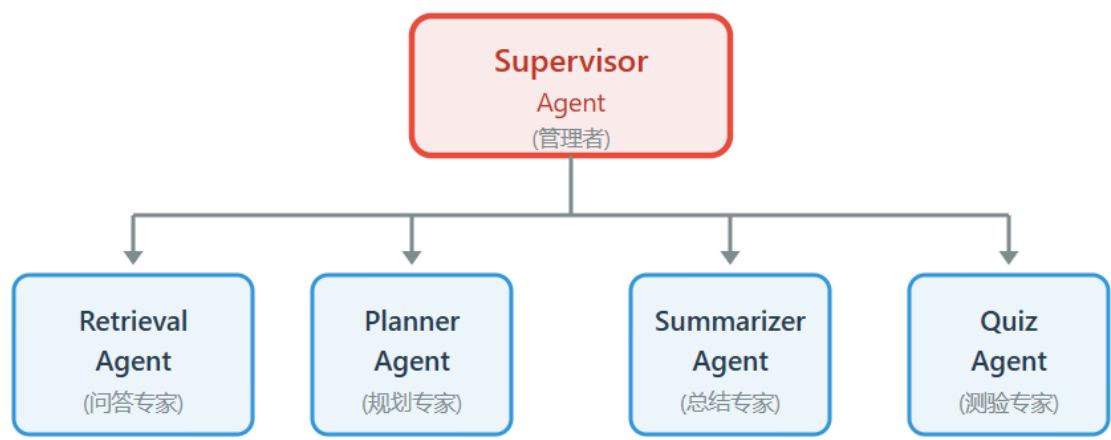
4.4 多智能体协作模式 (Multi-Agent Collaboration)

模式定义

多智能体协作即多个智能体共同实现目标。该模式利用专长分工、任务分布与智能体间通信。协作形式包括顺序交接、并行处理、辩论或层级结构。本系统采用**层级结构（监督者模式）**：由一个管理者智能体（Supervisor）根据能力动态分配任务给工作智能体，并综合结果。

在本系统中的应用

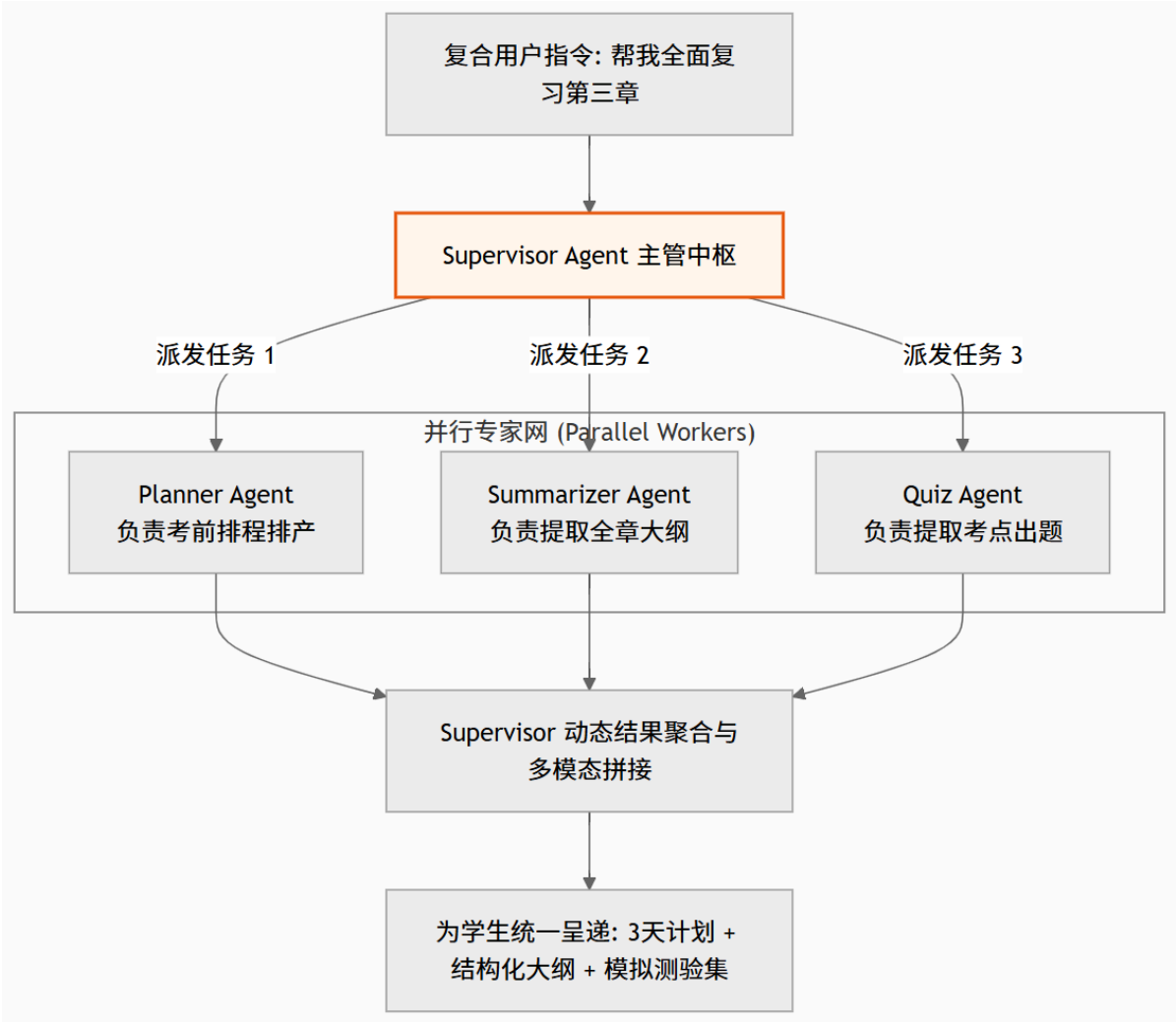
整体协作架构：



各Agent职责分工：

Agent	专长领域	核心能力	依赖工具
Supervisor Agent	意图理解与任务分配	分析用户意图，路由到合适的Agent	LLM意图分类
Retrieval Agent	基于知识的问答	RAG检索、答案生成、来源标注	ChromaDB、LLM
Planner Agent	学习计划生成	任务分解、时间规划、个性化建议	LLM Planning
Summarizer Agent	内容总结归纳	关键点提取、结构化笔记生成	Prompt Chain、LLM
Quiz Agent	题目生成与评估	知识点→题目转化、答案解析	LLM、知识库

协作流程（典型场景）：



作用与价值：

1. **专长分工：** 每个Agent只处理自己擅长的问题域
2. **并行处理：** 独立子任务可并发执行，提升响应速度
3. **可组合性：** 复杂用户请求可拆解为多Agent协作完成
4. **独立演进：** 每个Agent可独立升级模型或算法

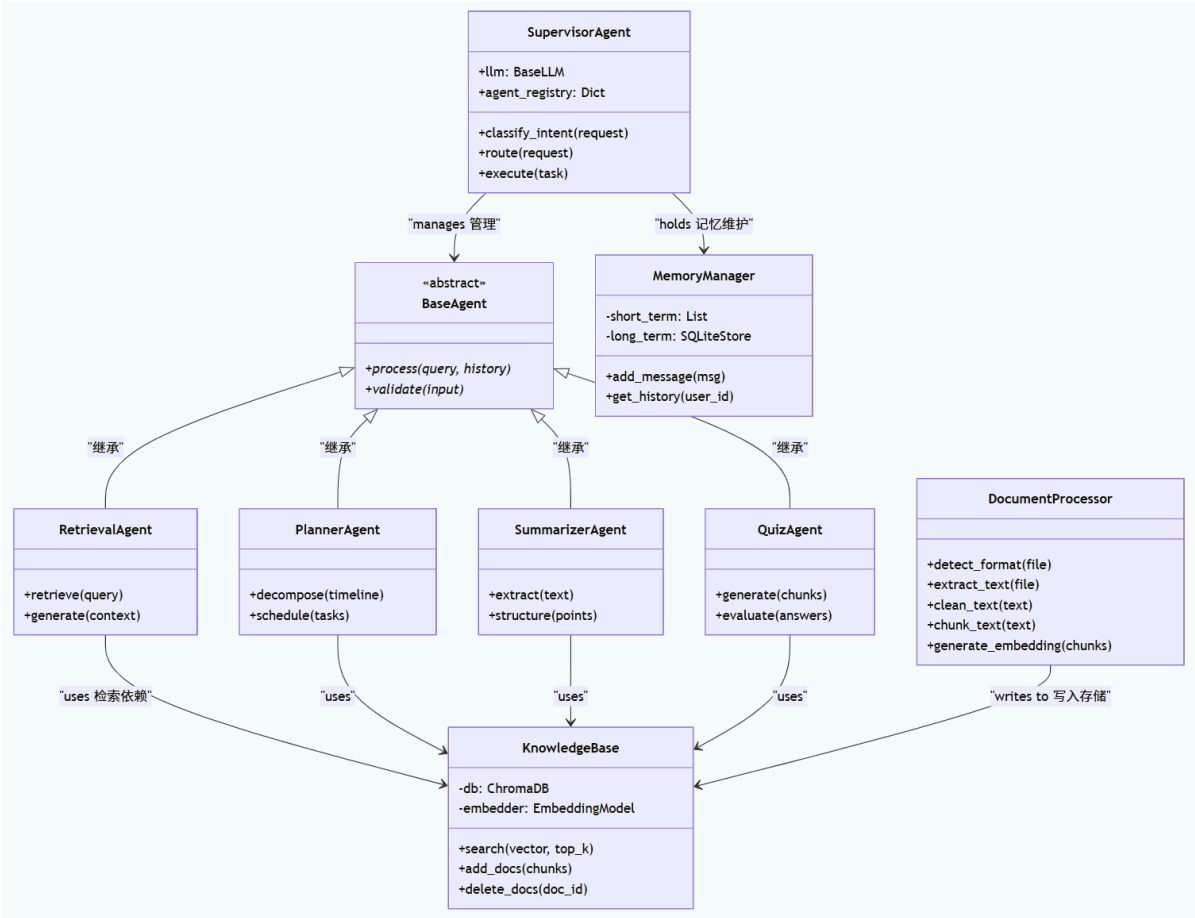
五、4+1 视图分析

5.1 逻辑视图 (Logical View)

关注点： 系统的功能需求——系统应为用户提供哪些服务。

核心组件/类结构：

系统采用面向对象设计，核心类及其关系如下：



5.2 开发视图 (Development View)

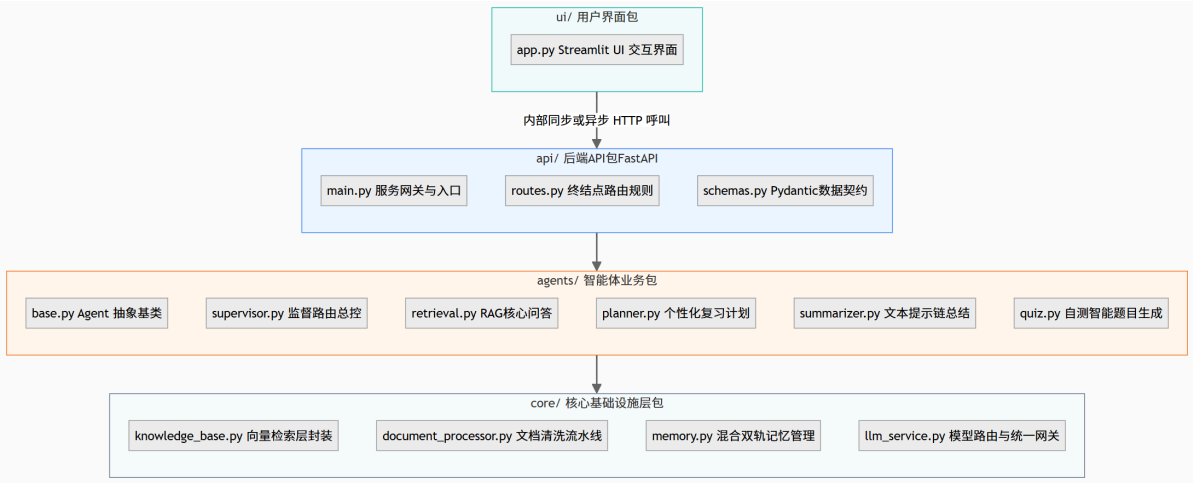
关注点：开发环境中的软件模块组织——代码结构、包划分、构建依赖。

模块划分：

```
软件体系结构大作业/
├─ src/
│   └─ agents/                                # Agent模块包
│       ├── __init__.py
│       ├── base.py                          # BaseAgent 抽象基类
│       ├── supervisor.py                    # SupervisorAgent - 意图路由
│       ├── retrieval.py                     # RetrievalAgent - RAG问答
│       ├── planner.py                       # PlannerAgent - 学习规划
│       ├── summarizer.py                   # SummarizerAgent - 知识总结
│       └── quiz.py                          # QuizAgent - 自测练习
│   └─ core/                                # 核心基础设施包
│       ├── __init__.py
│       ├── knowledge_base.py                # 知识库封装(ChromaDB)
│       ├── document_processor.py            # 文档处理流水线
│       ├── memory.py                       # 短期/长期记忆管理
│       ├── llm_service.py                  # LLM调用统一封装
│       └── config.py                       # 配置管理
│   └─ api/                                 # API服务层
│       ├── __init__.py
│       ├── main.py                         # FastAPI 应用入口
│       ├── routes.py                       # API路由定义
│       └── schemas.py                      # Pydantic数据模型
```

```
| | |
| | └─ ui/                                # 用户界面层
| |   └─ __init__.py
| |   └─ app.py                            # Streamlit 界面
|
| └─ data/                                # 数据存储目录
|   └─ chroma/                            # ChromaDB持久化数据
|   └─ uploads/                           # 上传的原始文件
|
| └─ requirements.txt                      # Python依赖
| └─ README.md                            # 项目说明
```

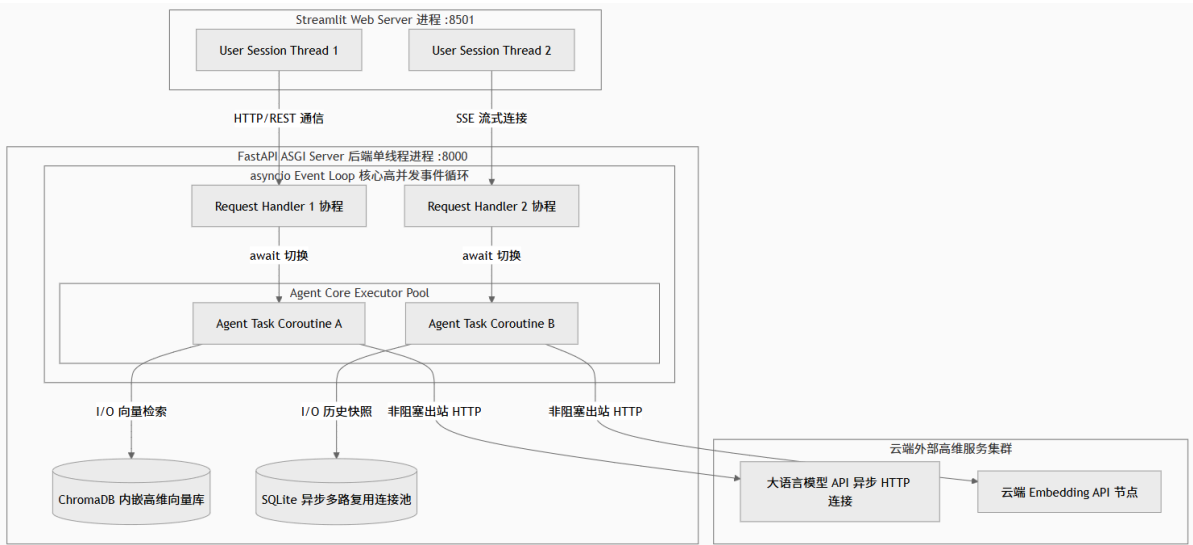
模块依赖关系：引用只能自顶向下，禁止循环引用



5.3 进程视图 (Process View)

关注点：系统的运行时行为——并发、通信、同步。

进程/线程模型：



并发机制：

层级	并发模型	通信方式	说明
前端 (Streamlit)	多线程（每Session一个线程）	HTTP Request	每个用户会话独立线程，共享全局Agent实例

层级	并发模型	通信方式	说明
后端 (FastAPI)	asyncio 异步事件循环	HTTP/SSE	单线程异步处理，避免GIL竞争
Agent执行	协程 (async/await)	Python asyncio	Agent内部LLM调用为异步I/O，多Agent可并发
文档处理	顺序流水线（可扩展为并行）	内存传递	当前为顺序执行，可扩展为message queue
数据库访问	异步连接池	线程安全API	ChromaDB使用HTTP客户端，SQLite使用aiosqlite

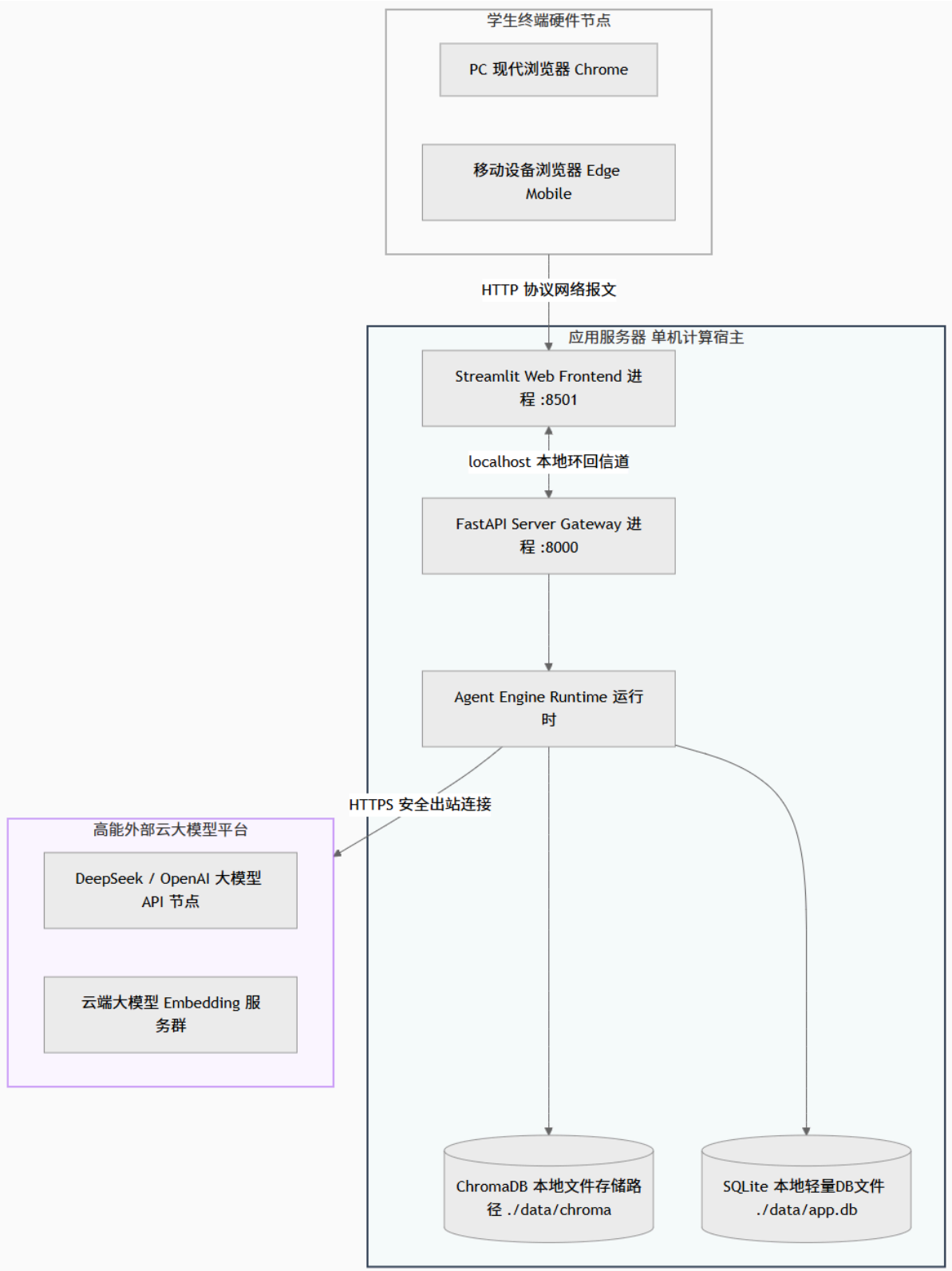
并发场景：

- **独立请求并发：**多个用户同时提问，FastAPI的asyncio事件循环自动调度
- **Agent间并行：**Supervisor可将独立子任务并行分配给多个Agent（如同时生成复习计划和知识总结）
- **文档处理串行：**文档处理流水线按序执行，保证数据一致性

5.4 物理视图 (Physical View)

关注点：系统的物理部署——软件到硬件的映射、网络拓扑。

部署架构：



部署说明:

组件	部署位置	硬件需求	网络
用户浏览器	用户终端设备	任意现代浏览器	访问 localhost:8501
Streamlit Frontend	应用服务器	与后端共享CPU/内存	绑定 0.0.0.0:8501
FastAPI Backend	应用服务器	CPU: 2核+, RAM: 4GB+	绑定 0.0.0.0:8000

组件	部署位置	硬件需求	网络
ChromaDB	应用服务器（嵌入式）	磁盘: 1GB+（随课件量增长）	进程内HTTP通信
SQLite	应用服务器（文件DB）	磁盘: 100MB+	本地文件I/O
LLM API	外部云服务	无需本地GPU	HTTPS出站
Embedding Model	本地或外部云服务	若本地: RAM 2GB+	本地加载或HTTPS

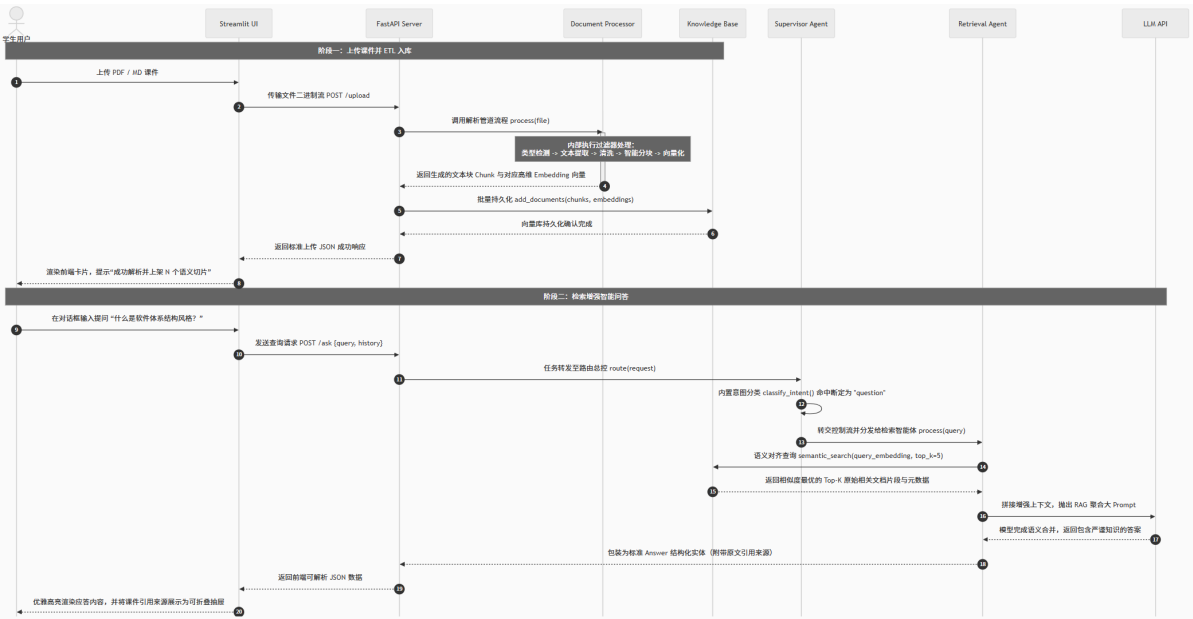
扩展性考虑：

- **Demo阶段：**所有组件单机部署，适合演示和个人使用
- **扩展方案：**ChromaDB可独立部署为服务；Agent可分布到不同服务器；可引入Redis做消息队列

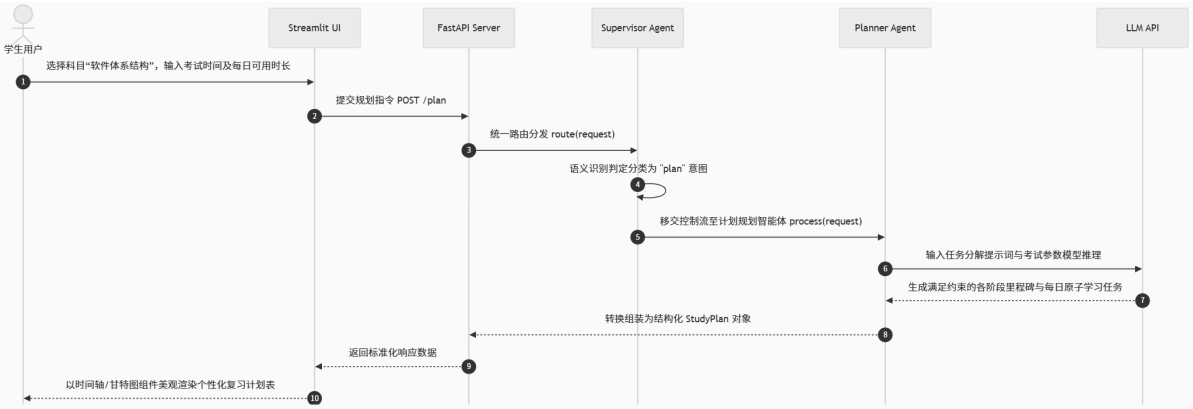
5.5 场景视图 (Scenario View, +1)

关注点：关键用例的端到端流程——将以上四个视图串联起来。

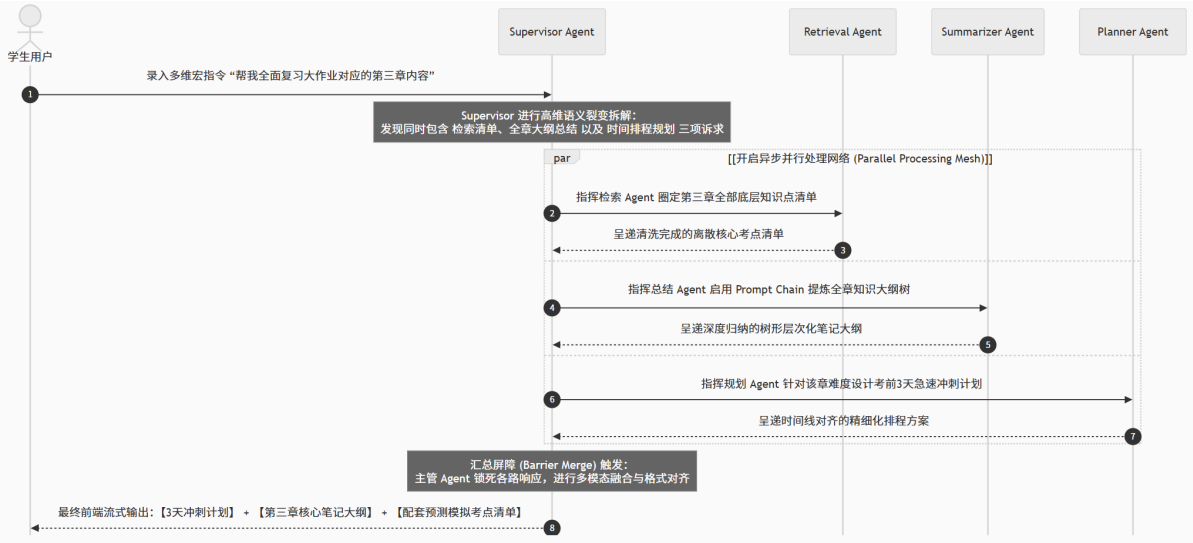
场景1：上传课件并提问（核心场景）



场景2：生成复习计划（规划场景）



场景3：多Agent并行协作场景



六、Demo 系统实现说明

6.1 技术栈

层次	技术选型	版本
前端	Streamlit	≥1.28
后端API	FastAPI + Uvicorn	≥0.104
LLM框架	LangChain	≥0.1.0
向量数据库	ChromaDB	≥0.4.0
嵌入模型	sentence-transformers (all-MiniLM-L6-v2)	≥2.2.0
关系存储	SQLite (aiosqlite)	≥0.19.0
文档解析	PyPDF2, python-docx, markdown	-

6.2 运行方式

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 启动后端 (终端1)
cd src/api
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# 3. 启动前端 (终端2)
cd src/ui
streamlit run app.py

# 4. 打开浏览器访问 http://localhost:8501
```

6.3 项目代码结构

参见 [5.2 开发视图](#)。

七、个人分工说明

本作业为**个人独立完成**，分工如下：

姓名	学号	负责模块	承担权重
李宗杭	2311451	全部模块：需求分析、体系结构设计、设计模式分析、4+1视图建模、Demo开发、文档撰写	100%